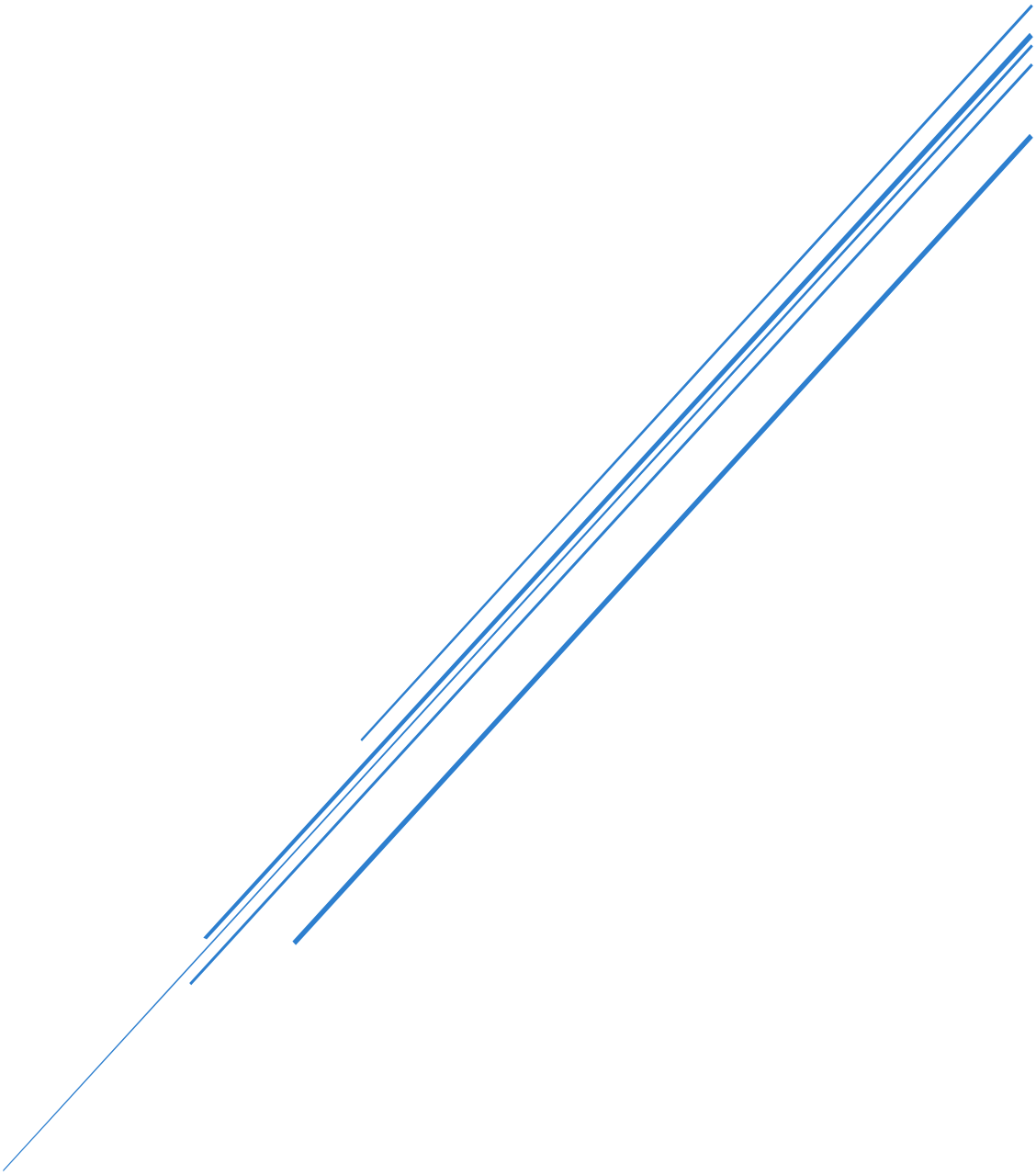
AWS2 10

Roberto Ripoll Cones

01/11/2026



IES Juan de Garay

CE CLOUD – Administración de redes en la nube

Contenido

1. Python 2
1.1. Creación de la VPC 2
1.2. Creación de la subred 2
1.3. Creación del grupo de seguridad 3
1.4. Obtención de AMI disponible 4
1.5. Creación e inicio de EC2 5
1.6. Ejecución del script 6
2. Bash 7
2.1. Creación de VPC 7
2.2. Creación de subred 7
2.3. Creación de grupo de seguridad 8
2.4. Obtención de AMI 9
2.5. Creación y ejecución de instancia 9
2.6. Ejecución de script 10

1. Python

1.1. Creación de la VPC

```python
def create_vpc():

    response = ec2.create_vpc(CidrBlock=VPC_CIDR)

    vpc_id = response["Vpc"]["VpcId"]

    ec2.get_waiter("vpc_available").wait(VpcIds=[vpc_id])

    ec2.create_tags(
        Resources=[vpc_id],
        Tags=[{"Key": "Name", "Value": "VPC_Practica"}]
    )

    print("VPC creada:", vpc_id)

    return vpc_id
```

1.2. Creación de la subred

```python
def create_subnet(vpc_id):

    response = ec2.create_subnet(
        VpcId=vpc_id,
        CidrBlock=SUBNET_CIDR
    )

    subnet_id = response["Subnet"]["SubnetId"]

    # Asignar IP pública automáticamente
    ec2.modify_subnet_attribute(
        SubnetId=subnet_id,
        MapPublicIpOnLaunch={"Value": True}
    )

    ec2.create_tags(
        Resources=[subnet_id],
        Tags=[{"Key": "Name", "Value": "Subred_Practica"}]
    )

    print("Subred creada:", subnet_id)

    return subnet_id
```

1.3. Creación del grupo de seguridad

```python
def create_security_group(vpc_id):

    response = ec2.create_security_group(
        GroupName="SG_Practica",
        Description="Grupo de seguridad para la EC2",
        VpcId=vpc_id
    )

    sg_id = response["GroupId"]

    # Permitir SSH, HTTP y HTTPS
    ec2.authorize_security_group_ingress(
        GroupId=sg_id,
        IpPermissions=[
            {
                "IpProtocol": "tcp",
                "FromPort": 22,
                "ToPort": 22,
                "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
            },
            {
                "IpProtocol": "tcp",
                "FromPort": 80,
                "ToPort": 80,
                "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
            },
            {
                "IpProtocol": "tcp",
                "FromPort": 443,
                "ToPort": 443,
                "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
            },
            {
                "IpProtocol": "icmp",
                "FromPort": 8,
                "ToPort": -1,
                "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
            }
        ]
    )

    print("Grupo de seguridad creado:", sg_id)

    return sg_id
```

1.4. Obtención de AMI disponible

```python
def get_ubuntu_ami():

    response = ec2.describe_images(
        Owners=["099720109477"],
        Filters=[
            {
                "Name": "name",
                "Values": [
                    "ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"
                ]
            },
            {
                "Name": "state",
                "Values": ["available"]
            }
        ]
    )

    images = response["Images"]

    if not images:
        raise Exception("No se ha encontrado una AMI de Ubuntu 22.04")

    latest_image = sorted(
        images,
        key=lambda image: image["CreationDate"]
    )[-1]

    return latest_image["ImageId"]
```

1.5. Creación e inicio de EC2

```python
def create_instance(subnet_id, sg_id):

    ami_id = get_ubuntu_ami()

    response = ec2.run_instances(
        ImageId=ami_id,
        InstanceType=INSTANCE_TYPE,
        KeyName=KEY_NAME,
        MinCount=1,
        MaxCount=1,

        SubnetId=subnet_id,
        SecurityGroupIds=[sg_id],

        IamInstanceProfile={
            "Name": IAM_PROFILE
        },

        BlockDeviceMappings=[
            {
                "DeviceName": "/dev/sda1",
                "Ebs": {
                    "VolumeSize": 10,
                    "VolumeType": "gp2",
                    "DeleteOnTermination": True
                }
            }
        ],

        TagSpecifications=[
            {
                "ResourceType": "instance",
                "Tags": [
                    {"Key": "Name", "Value": INSTANCE_NAME}
                ]
            }
        ]
    )

    instance_id = response["Instances"][0]["InstanceId"]

    print("Instancia EC2 creada:", instance_id)

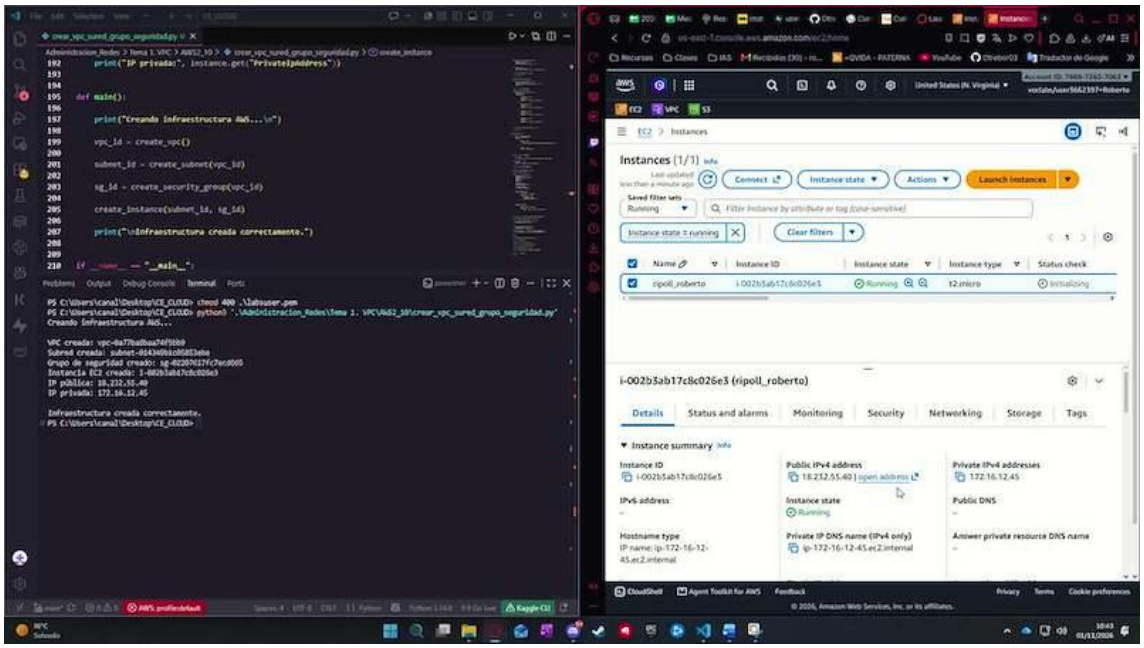
    ec2.get_waiter("instance_running").wait(
        InstanceIds=[instance_id]
    )

    instance = ec2.describe_instances(
        InstanceIds=[instance_id]
    )["Reservations"][0]["Instances"][0]

    print("IP pública:", instance.get("PublicIpAddress"))
    print("IP privada:", instance.get("PrivateIpAddress"))

```

1.6. Ejecución del script



Video Python

Al ejecutar el script de Python, podemos comprobar que se crean correctamente todos los recursos definidos: la VPC, la subred, el grupo de seguridad y la instancia EC2. Además, la instancia recibe una dirección IP privada y otra pública.

Sin embargo, al no disponer de un Internet Gateway asociado a la VPC ni de una ruta hacia Internet, no es posible establecer una comunicación directa desde nuestro equipo externo hacia la instancia. Por este motivo, las pruebas realizadas mediante ping y SSH no obtienen respuesta.

Por tanto, podemos verificar que la infraestructura se ha creado correctamente, pero no comprobar la conectividad desde el exterior sin configurar previamente los elementos de conexión a Internet.

2. Bash

2.1. Creación de VPC

```
22  create_vpc() {
23
24      VPC_ID=$(aws ec2 create-vpc \
25          --cidr-block "$VPC_CIDR" \
26          --query "Vpc.VpcId" \
27          --output text)
28
29      aws ec2 wait vpc-available --vpc-ids "$VPC_ID"
30
31      aws ec2 create-tags \
32          --resources "$VPC_ID" \
33          --tags Key=Name,Value=VPC_Practica
34
35      echo "VPC creada: $VPC_ID"
36  }
37
38
```

2.2. Creación de subred

```
40  create_subnet() {
41
42      SUBNET_ID=$(aws ec2 create-subnet \
43          --vpc-id "$VPC_ID" \
44          --cidr-block "$SUBNET_CIDR" \
45          --query "Subnet.SubnetId" \
46          --output text)
47
48      aws ec2 wait subnet-available --subnet-ids "$SUBNET_ID"
49
50      # Asignar IP publica automaticamente
51      aws ec2 modify-subnet-attribute \
52          --subnet-id "$SUBNET_ID" \
53          --map-public-ip-on-launch
54
55      aws ec2 create-tags \
56          --resources "$SUBNET_ID" \
57          --tags Key=Name,Value=Subred_Practica
58
59      echo "Subred creada: $SUBNET_ID"
60  }
```

2.3. Creación de grupo de seguridad

```
create_security_group() {

    SG_ID=$(aws ec2 create-security-group \
        --group-name SG_Practica \
        --description "Grupo de seguridad para la EC2" \
        --vpc-id "$VPC_ID" \
        --query "GroupId" \
        --output text)

    # Reglas SSH, HTTP, HTTPS y PING (ICMP)
    IP_PERMISSIONS='[
        {
            "IpProtocol": "tcp",
            "FromPort": 22,
            "ToPort": 22,
            "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
        },
        {
            "IpProtocol": "tcp",
            "FromPort": 80,
            "ToPort": 80,
            "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
        },
        {
            "IpProtocol": "tcp",
            "FromPort": 443,
            "ToPort": 443,
            "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
        },
        {
            "IpProtocol": "icmp",
            "FromPort": 8,
            "ToPort": -1,
            "IpRanges": [{"CidrIp": "0.0.0.0/0"}]
        }
    ]'

    aws ec2 authorize-security-group-ingress \
        --group-id "$SG_ID" \
        --ip-permissions "$IP_PERMISSIONS"

    aws ec2 create-tags \
        --resources "$SG_ID" \
        --tags Key=Name,Value=SG_Practica

    echo "Grupo de seguridad creado: $SG_ID"
}

```

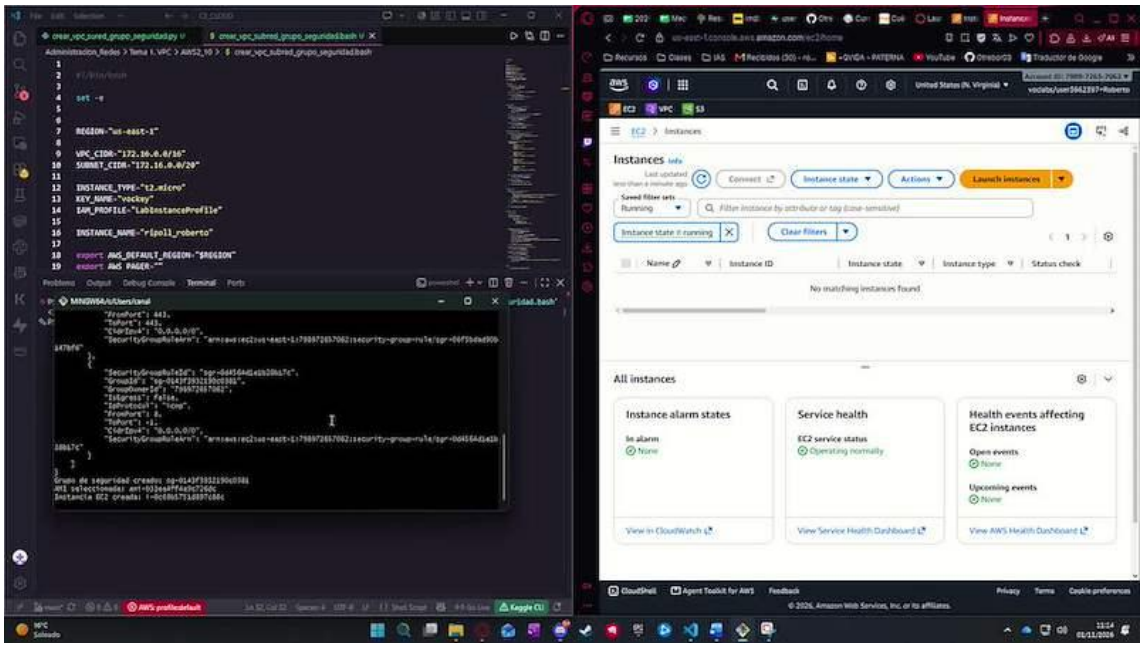
2.4. Obtención de AMI

```
111  get_ubuntu_ami() {
112
113      AMI_ID=$(aws ec2 describe-images \
114          --owners 099720109477 \
115          --filters \
116              "Name=name,Values=ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*" \
117              "Name=state,Values=available" \
118          --query 'Images | sort_by(@, &CreationDate)[-1].ImageId' \
119          --output text)
120
121      if [ -z "$AMI_ID" ] || [ "$AMI_ID" = "None" ]; then
122          echo "Error: No se ha encontrado una AMI de Ubuntu 22.04"
123          exit 1
124      fi
125
126      echo "AMI seleccionada: $AMI_ID"
127  }
128
```

2.5. Creación y ejecución de instancia

```
130  create_instance() {
131
132      get_ubuntu_ami
133
134      BLOCK_DEVICE_MAPPINGS='[
135          {
136              "DeviceName": "/dev/sda1",
137              "Ebs": {
138                  "VolumeSize": 10,
139                  "VolumeType": "gp2",
140                  "DeleteOnTermination": true
141              }
142          }
143      ]'
144
145      INSTANCE_ID=$(aws ec2 run-instances \
146          --image-id "$AMI_ID" \
147          --instance-type "$INSTANCE_TYPE" \
148          --key-name "$KEY_NAME" \
149          --count 1 \
150          --iam-instance-profile "Name=$IAM_PROFILE" \
151          --subnet-id "$SUBNET_ID" \
152          --security-group-ids "$SG_ID" \
153          --block-device-mappings "$BLOCK_DEVICE_MAPPINGS" \
154          --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$INSTANCE_NAME}]" \
155          --query "Instances[0].InstanceId" \
156          --output text)
157
158      echo "Instancia EC2 creada: $INSTANCE_ID"
159
160      # Esperar a que la instancia este en ejecucion
161      aws ec2 wait instance-running \
162          --instance-ids "$INSTANCE_ID"
163
164      # Esperar a las comprobaciones 2/2
165      aws ec2 wait instance-status-ok \
166          --instance-ids "$INSTANCE_ID"
167
168      # Mostrar informacion de la instancia
169      aws ec2 describe-instances \
170          --instance-ids "$INSTANCE_ID" \
171          --query "Reservations[0].Instances[0].[State.Name,PublicIpAddress,PrivateIpAddress]" \
172          --output table
173  }
174
```

2.6. Ejecución de script



Video Bash